

CS5344 Project Proposal

Steering a Tabular Diffusion Model Away from Attacks

Synthetic normal traffic for anomaly detection on NSL-KDD and UNSW-NB15

Group 09

Name	Student number
Brian Kheng	A0290248B
Mei Xianle	A0000000X
Passion Goh Wei Ling	A0000000X

27 September 2026

1 Introduction

For each of two network intrusion datasets we get a labelled training set and have to submit a fixed number of synthetic normal rows: 40,000 for NSL-KDD and 70,000 for UNSW-NB15. The organisers fit two fixed detectors on those rows, ECOD and an Isolation Forest (10 seeds of 500 trees), and score them by the area under the precision-recall curve (AUPRC) on hidden validation and test sets. The final score is the mean of the two AUPRCs, and the hidden sets contain attack types that never appear in training.

This makes the task a bit unusual. We never train a detector ourselves, and the rows we submit are, as far as the detectors know, the definition of normal traffic. A good submission has to cover the normal traffic in the hidden sets, so that it isn't flagged, while leaving the places where attacks live empty, so that attacks land in the tails. Copying the training distribution does the first job but not the second.

Our plan is to train a mixed-type diffusion model, TabDiff [12], on all training rows with the label as an extra column, and then sample normal rows with a guidance term that pushes each sample away from what the model associates with attacks. Section 2 looks at where the attacks actually sit in the data, Section 3 covers related work, and Section 4 describes the method, the evidence for it so far, and how we will evaluate it.

2 Exploratory data analysis

Table 1 summarises the two training sets. Both are mixed-type tables with three categorical columns, and both have a little over twice as many normal rows as attacks. The numeric columns are hard to model. In NSL-KDD, 23 of the 38 numeric columns are zero in over 90% of normal rows, and `src_bytes` has a skewness of about 129 among normal rows. Eleven NSL-KDD columns and ten UNSW-NB15 columns take at most 20 distinct values (flags, TTLs and small counters), and two NSL-KDD columns, `num_outbound_cmds` and `is_host_login`, are constant. A generator that treats all of these as continuous will waste probability mass on values that never occur, such as a TTL of 45.

Table 1: The two training sets. Ranges and categories are taken from the normal training rows. Computed by `proposal/eda.py`.

	NSL-KDD	UNSW-NB15
Training rows (normal / attack)	20,000 / 10,000	35,000 / 15,000
Required synthetic rows N	40,000	70,000
Numeric / categorical features	38 / 3	39 / 3
Numeric features with at most 20 distinct values	11	10
Constant features	2	0
Numeric features that are zero in over 90% of normal rows	23	4
Attacks with a category no normal row uses	32.5%	1.7%
Attacks with a numeric value outside the normal range	2.0%	1.1%
Attacks inside the normal range on every feature	65.5%	97.2%
Numeric features with normal-vs-attack KS distance above 0.5	13	3

The briefing distinguishes anomalies with an odd value from anomalies that are a rare combination of ordinary values, so we checked which kind the training attacks are. For each attack row we asked whether any numeric value falls outside the range seen in normal rows, or whether any categorical value never appears in a normal row. In NSL-KDD, 32.5% of attacks use a category that no normal row uses (41 of the 67 `service` values appear only in attacks), but only 2.0% have an out-of-range number. The other 65.5% sit inside the normal range on every column. UNSW-NB15 is far more extreme: 97.2% of its attacks are inside the normal range everywhere.

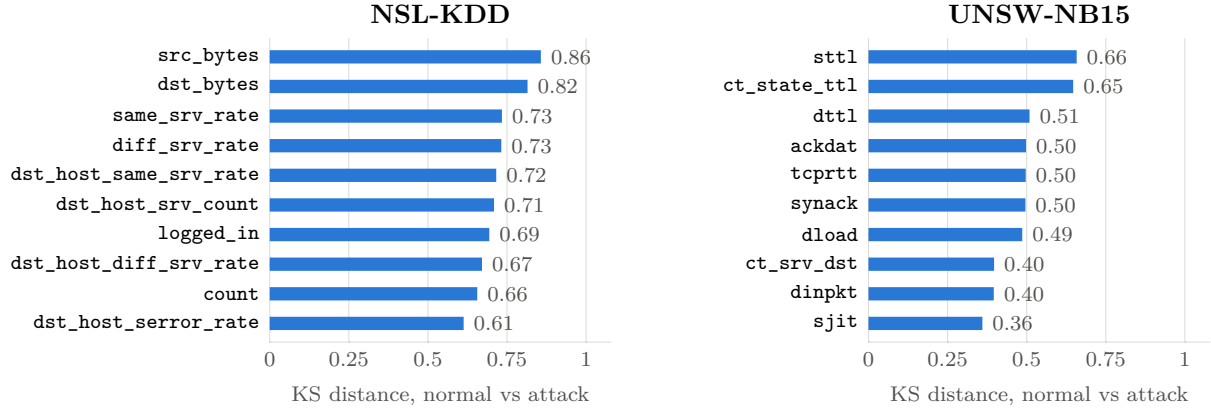


Figure 1: The ten numeric columns of each training set whose normal and attack values differ most, measured by the two-sample Kolmogorov-Smirnov (KS) distance. Separation in UNSW-NB15 is weaker and concentrated in the TTL columns.

Figure 1 ranks the numeric columns by the Kolmogorov-Smirnov (KS) distance between their normal and attack values. NSL-KDD has 13 columns above 0.5, led by the byte counts. UNSW-NB15 has three, all derived from TTLs, and these show how an attack can look unremarkable one column at a time. Every UNSW-NB15 attack has `sttl` equal to 254 (71.5%) or 62 (28.5%), and both values are common in normal traffic too, at 27.1% and 7.0% of normal rows. The attacks give themselves away through how often these values occur and what they occur together with, not through the values themselves.

This matters because of how the two detectors work. ECOD scores each column by its tail probability under the empirical distribution of the training rows and adds the scores up, so it only responds to odd values. Isolation Forest can isolate rare combinations with its random axis-parallel splits, but with scikit-learn’s default settings each tree only sees 256 rows. Both are fitted on our submission. So on UNSW-NB15, the only way to raise an attack’s ECOD score is to make values such as `sttl` = 254 rarer in the synthetic data than they are in real normal traffic, and a faithful copy of the normal rows cannot do that. This fits the low scores on this dataset: the provided sample submission scores 0.383 on the public leaderboard for UNSW-NB15, against 0.899 for NSL-KDD.

3 Related work

3.1 Unsupervised anomaly detection

Chandola et al. [1] survey the classical anomaly detection methods and Pang et al. [11] the deep ones. The two detectors used here are fast, unsupervised methods for tabular data. Isolation Forest [9] builds random trees on small subsamples and scores a point by how few random splits it takes to isolate it, so it responds to low density without needing a distance metric. ECOD [8] estimates the empirical cumulative distribution function of each feature and scores a point by its tail probabilities, summed over features. It has no parameters and is easy to interpret, but it ignores the dependence between features, which is exactly the weakness Section 2 points at.

ADBench [5] compared 30 detectors on 57 datasets and found that no unsupervised method was significantly better than the others overall. Which one wins depends on whether its assumptions match the type of anomaly. In this project the detectors are fixed, so we can’t pick a method to suit the anomalies. What we control is the training data, the other half of that match.

3.2 Tabular data synthesis

The simplest augmentation is interpolation. SMOTE [2] creates new rows on the line between a row and one of its nearest neighbours. It is cheap, but it only fills in between existing rows and has no natural way to handle categorical columns. CTGAN and TVAE [13] model mixed-type tables with a GAN and a variational autoencoder, using mode-specific normalisation for multi-modal numeric columns and a conditional generator so that rare categories still get sampled. TabDDPM [7] brought diffusion models to tables, with Gaussian diffusion for numeric columns and multinomial diffusion for categorical ones. TabSyn [14] instead runs score-based diffusion in the latent space of a VAE. TabDiff [12] trains a single continuous-time diffusion over both types, with masking diffusion for categorical columns and a learnable noise schedule for every column, and reports better fidelity than these earlier models on the benchmarks it tests.

These papers judge synthetic data mainly by fidelity, meaning how closely its marginals and correlations match the real data, and by utility: train a model on synthetic data and test it on real data, a protocol usually traced back to Esteban et al. [4]. Our task is a utility test where the model is fixed to two unsupervised detectors. That changes the goal, since the most faithful submission is not necessarily the most useful one.

3.3 Guidance in diffusion models

A conditional diffusion model can be steered while sampling. Classifier guidance [3] adds the gradient of a separately trained classifier to the model’s score. Classifier-free guidance [6] drops the classifier: one network is trained with and without the condition, and sampling extrapolates from the unconditional prediction towards the conditional one, trading some diversity for samples that fit the condition more strongly. Liu et al. [10] showed that the same arithmetic can combine several conditions, including negated ones, where subtracting the prediction for an unwanted concept moves samples away from it. This work was done on images and text prompts. We haven’t found prior work that uses guidance to push tabular samples away from a class in order to help anomaly detectors, and that is the gap our method targets.

4 Intended approach

Figure 2 shows the generator. It is one pipeline for both datasets, as the rules require, with only the guidance weight w , the training length and N set per dataset.

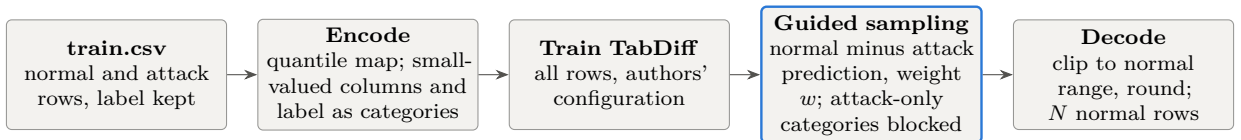


Figure 2: The generator. The outlined step is our addition to TabDiff.

4.1 Backbone

We use the TabDiff authors’ model code and default configuration unchanged, and write the data handling, training loop and sampling ourselves. Before training, numeric columns with at most 20 distinct values are treated as categories, so the model can only produce values that actually occur, and constant columns are held fixed. Continuous columns are mapped to a standard normal with a quantile transform, following the TabDDPM and TabDiff preprocessing, which deals with the heavy tails from Section 2. The label is added as one more categorical column and the model is trained on every training row, attacks included, so that it learns what attacks look like as well as normal traffic. The guidance needs both.

Training follows the authors’ trainer: 8,000 epochs of AdamW with batch size 4,096, a plateau learning-rate decay, a continuous-loss weight that is annealed over training, and an exponential moving average of the weights, from which we keep the copy with the lowest training loss in the second half of training.

4.2 Contrastive guidance

At every sampling step we run TabDiff’s denoiser D twice on the current noisy row x , once with the label set to normal and once with it set to attack, and combine the two outputs:

$$\tilde{D}(x) = D(x \mid \text{normal}) + w (D(x \mid \text{normal}) - D(x \mid \text{attack})). \quad (1)$$

The same combination is applied to the logits of the categorical columns. With $w = 0$ this is plain label-conditional TabDiff. With $w > 0$ it is classifier-free guidance with the attack condition in place of the unconditional one, which is a negated condition in the sense of Liu et al. [10].

The rules ask us to explain the distribution of the generated data, and (1) gives a direct answer. The model’s score is a linear function of the denoiser output, so (1) follows the score of

$$p_w(x) \propto p(x \mid \text{normal}) \left(\frac{p(x \mid \text{normal})}{p(x \mid \text{attack})} \right)^w \quad (2)$$

at each noise level. The samples follow the normal distribution, reweighted by how strongly each region favours normal over attack. Regions only normal traffic uses keep their mass or gain some, while regions shared with attacks, such as `stt1` = 254 in UNSW-NB15, are thinned out, and w sets how hard. As with classifier-free guidance, (2) is exact only for the noisy intermediate distributions, not for the final samples, so we also measure the final samples directly (Section 4.4).

Two further constraints keep every sample a valid normal row. Categories that no normal training row uses, including the attack label itself, are blocked during sampling by setting their logits to $-\infty$, so, for example, none of the 41 attack-only NSL-KDD services ever appears in the submission. After sampling, continuous values are clipped to the range seen in normal rows and rounded to each column’s precision, which keeps the output within the submission’s types and ranges.

4.3 Why this design

Before settling on TabDiff we compared five generators with our local judge (Section 4.4). Table 2 shows the results. On NSL-KDD no generator beats a plain bootstrap of the normal rows by more than 0.0022. TabDiff comes closest, and its samples are by far the most faithful, with a mean per-column KS distance to the normal rows of 0.014 against 0.08 to 0.14 for the other generators. TabDDPM’s published sampler diverged on NSL-KDD, so its entry uses a version that clamps the predicted values while sampling.

Table 2: Preliminary comparison of generators trained on the normal training rows with their published settings. Local combined AUPRC, averaged over two disjoint 20,000-row samples of the public datasets that exclude all coursework training rows. The first two rows are averaged over five seeds (standard deviations 0.0003 to 0.0014); the generative models ran once.

Generator	NSL-KDD	UNSW-NB15
Bootstrap of normal rows	0.9334	0.5962
Bootstrap plus hand-written per-column reshaping	0.9327	0.6515
CTGAN	0.9351	0.5513
TVAE	0.9266	0.5918
TabDDPM	0.8910	0.5713
TabDiff	0.9356	not finished

On UNSW-NB15 the choice of generator matters much less than whether the labels are used. Our earlier generator, a bootstrap followed by a hand-written per-column reshaping away from attack values, gains 0.055 over the bootstrap. Applying the same reshaping to the CTGAN, TVAE and TabDDPM samples lifts them to between 0.629 and 0.643. This is what the EDA predicts: when attacks share their values with normal traffic, the synthetic data has to be shaped by where the attacks are. Contrastive guidance is a learned version of that hand-written reshaping, acting on all columns jointly instead of one at a time.

We have also checked that the guidance pushes in the right direction. With a model trained for only 300 epochs on NSL-KDD, a normal-versus-attack classifier fitted on the training data gave the samples a mean attack probability of 0.317 at $w = 0$, 0.201 at $w = 1$ and 0.130 at $w = 2$. That confirms the direction of the push but says nothing yet about AUPRC. Full 8,000-epoch runs at $w \in \{0, 1, 2\}$ on both datasets are in progress.

4.4 Evaluation plan

We reimplemented the scoring described in the briefing as a local judge: ECOD from PyOD and scikit-learn’s Isolation Forest with 500 trees for each of the seeds 0 to 9, both fitted on the submitted rows with one-hot encoded categories, and the mean of the ECOD AUPRC and the average Isolation Forest AUPRC. Some details are unspecified, such as the Isolation Forest subsample size and the categorical encoding, so we record our choices and treat the judge as a proxy. On the sample submissions it gives 0.903 for NSL-KDD, close to the public leaderboard’s 0.899, but 0.664 for UNSW-NB15 against 0.383. On UNSW-NB15 we will therefore use it to compare methods, not to predict the leaderboard.

The results in Table 2 use rows from the public datasets. The organisers built their hidden sets from the same public data, so these rows may overlap them, and we won’t use them to choose hyperparameters. To tune w we will hold out a stratified 20% of `train.csv`, train the generator on the other 80%, and score on the holdout. The public data then serves only as a final check.

Guidance pushes away from the attacks the model has seen, while the hidden test set has types it hasn’t. The attack type isn’t released, so we will simulate unseen types by clustering the training attacks with k -means on the encoded features, leaving one cluster out of generator training at a time, and measuring AUPRC on the held-out cluster. If guidance helps on seen attacks but hurts on held-out clusters, we will lower w or mix guided and unguided samples.

For every run we will report the two detector AUPRCs separately, per-column KS and total variation distances to the normal training rows, and the classifier attack probability above, so that score changes can be explained in terms of the data. The ablations are $w \in \{0, 0.5, 1, 2, 4\}$, sampling with and without blocked categories, and the bootstrap baseline. We will submit to Kaggle sparingly, mainly to confirm that local improvements carry over, since the public leaderboard only reflects the validation attack types.

The main risk is over-guidance. A large w also removes normal traffic that happens to look like attacks, and those normal rows in the hidden set then score as anomalies, which costs precision. The holdout and held-out-cluster experiments are there to find where that starts. Training is also slow, about two hours for NSL-KDD on a laptop GPU and longer for UNSW-NB15, but the trained model is cached, so each new value of w only reruns sampling, which takes minutes.

References

- [1] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), 2009.
- [2] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE:

- Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 2002.
- [3] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat GANs on image synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
 - [4] Cristóbal Esteban, Stephanie L. Hyland, and Gunnar Rätsch. Real-valued (medical) time series generation with recurrent conditional GANs. arXiv:1706.02633, 2017.
 - [5] Songqiao Han, Xiyang Hu, Hailiang Huang, Minqi Jiang, and Yue Zhao. ADBench: Anomaly detection benchmark. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.
 - [6] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. arXiv:2207.12598, 2022.
 - [7] Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev, and Artem Babenko. TabDDPM: Modelling tabular data with diffusion models. In *International Conference on Machine Learning (ICML)*, 2023.
 - [8] Zheng Li, Yue Zhao, Xiyang Hu, Nicola Botta, Cezar Ionescu, and George H. Chen. ECOD: Unsupervised outlier detection using empirical cumulative distribution functions. *IEEE Transactions on Knowledge and Data Engineering*, 2022.
 - [9] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *IEEE International Conference on Data Mining (ICDM)*, 2008.
 - [10] Nan Liu, Shuang Li, Yilun Du, Antonio Torralba, and Joshua B. Tenenbaum. Compositional visual generation with composable diffusion models. In *European Conference on Computer Vision (ECCV)*, 2022.
 - [11] Guansong Pang, Chunhua Shen, Longbing Cao, and Anton van den Hengel. Deep learning for anomaly detection: A review. *ACM Computing Surveys*, 54(2), 2021.
 - [12] Juntong Shi, Minkai Xu, Harper Hua, Hengrui Zhang, Stefano Ermon, and Jure Leskovec. TabDiff: a mixed-type diffusion model for tabular data generation. In *International Conference on Learning Representations (ICLR)*, 2025.
 - [13] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. Modeling tabular data using conditional GAN. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
 - [14] Hengrui Zhang, Jiani Zhang, Balasubramaniam Srinivasan, Zhengyuan Shen, Xiao Qin, Christos Faloutsos, Huzefa Rangwala, and George Karypis. Mixed-type tabular data synthesis with score-based diffusion in latent space. In *International Conference on Learning Representations (ICLR)*, 2024.